

135. Se souvenir d'une partie à l'autre

Un meilleur score qui survit à l'extinction.

1. Ce qu'il faut comprendre

La cartouche porte une pile et une petite mémoire. `sauver(numero, valeur)` y écrit, `sauvegarde(numero)` y relit — 256 cases, et elles sont encore là après avoir éteint la console.

Cette mémoire n'est pas ouverte en permanence : il faut la **déverrouiller** avant d'y toucher et la refermer aussitôt. Le compilateur pose le verrou lui-même à chaque accès. Ce n'est pas de la prudence excessive : laissée ouverte, une coupure de courant au mauvais moment la corrompt.

À la toute première partie, cette mémoire contient n'importe quoi. Un jeu y écrit donc **une marque à lui** et ne fait confiance au reste que s'il la retrouve. C'est ce que fait la première ligne de `main()`.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <sauvegarde>` → [119.2. La fonction sauvegarde\(\) — relire un nombre gardé](#)
- `#include <sauver>` → [119.1. La fonction sauver\(\) — garder un nombre, même éteint](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1094 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      58 o ← la console, à l'allumage — la mise en route de la console
main()         215 o ← démarrage — ta fonction
VBlank         8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage  28 o ← image() — attend l'image suivante
AttendreVBlank 16 o ← démarrage — attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte — attend de pouvoir écrire une case
EcrireTexte    10 o ← texte() — écrit un texte, lettre par lettre
CopierTuiles   18 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage — vide les deux cartes de l'écran
Tuiles         704 o ← CopierTuiles — la police entière
"SCORE"        5 o ← texte() — un texte gravé
"RECORD"       6 o ← texte() — un texte gravé
```

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `AdresseCase`, `EffacerCases`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
sauver()      x 3    73 o sur place, dans main()
sauvegarde() x 2    41 o sur place, dans main()
poser()       x 2    24 o sur place, dans main()
texte()       x 2    22 o sur place, dans main() + EcrireTexte (10 o)
image()       x 1     3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <sauvegarde>    // relit un nombre gardé dans la cartouche
#include <sauver>        // garde un nombre dans la cartouche, même éteinte
#include <texte>         // écrit un texte à l'écran
#include <poser>         // pose une tuile sur une case du fond
```

2. UNE CONSTANTE — MARQUE

```
const uint8_t MARQUE = 42;
```

3. UNE VARIABLE — MEILLEUR

```
uint8_t meilleur = 0;
```

4. UNE VARIABLE — SCORE

```
uint8_t score = 0;
```

5. UNE VARIABLE — ATTENTE

```
uint8_t attente = 0;
```

6. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // La cartouche a-t-elle déjà servi ?
    if (sauvegarde(0) != MARQUE) {
        sauver(0, MARQUE);
        sauver(1, 0);
    }

    meilleur = sauvegarde(1);

    texte(2, 4, "SCORE");
    texte(2, 8, "RECORD");

    while (true) {
        image();

        attente++;

        if (attente > 30) {
            attente = 0;

            if (score < 9) score++;

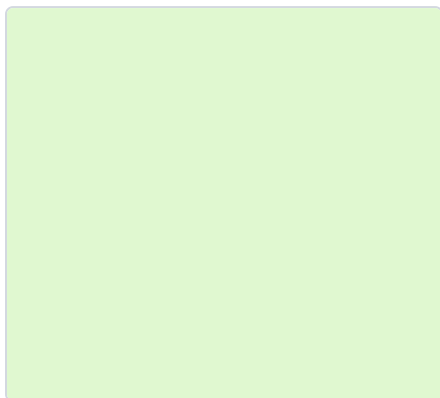
            if (score > meilleur) {
                meilleur = score;
                sauver(1, meilleur);
            }
        }

        poser(10, 4, 27 + score);
        poser(10, 8, 27 + meilleur);
    }

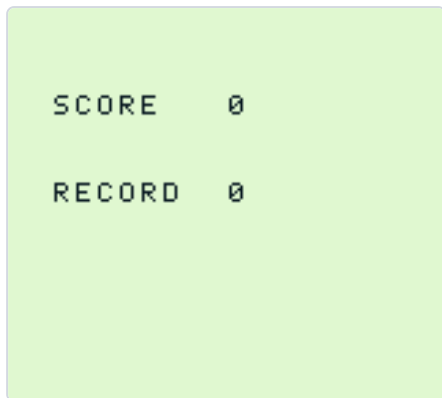
    return 0;
}
```

3. Ce qui se passe, étape par étape

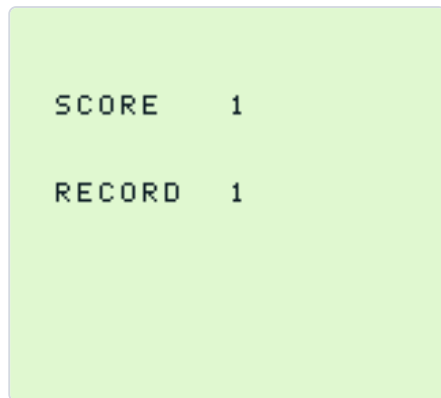
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



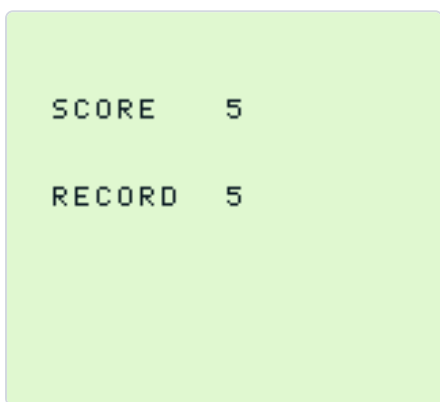
la toute première image — la console vient de démarrer — image 1



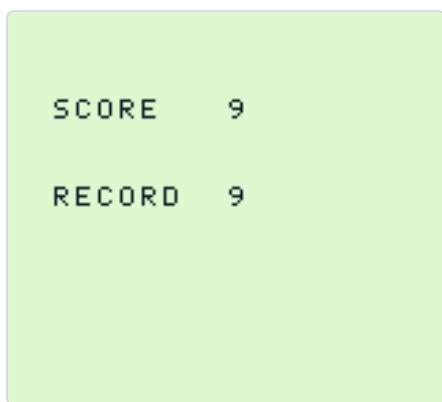
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Le score monte, et le record le suit — puis lui survit.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la cartouche annonce une mémoire à pile
- ✓ la marque est écrite
- ✓ le score monte (7)
- ✓ le record le suit (7)
- ✓ et il est bien dans la mémoire de la cartouche
- ✓ la mémoire est refermée après chaque accès